FACULTY OF COMPUTERS AND ARTIFICIAL INTELLIGENCE
CAIRO UNIVERSITY
Department of Artificial Intelligence

PROJECT DOCUMENTATION 2024

AUDIFY

EFFORTLESS AUDIO PERFECTION
EDIT WITH EASE, SPEAK WITH CONFIDENCE

AUDIFY MEMBERS

<Moustafa Omar Mohammed>
<20200542>

<Abdelrahman Ramadan Aboulela>
<20200284>

<Ahmed Hany Ibrahim>
<20200054>

<Ezz El-Din Ahmed Saber>
<20200325>

<Marwa Shaaban Aid >
<20200516>

Project Advisor

Dr.Farid Ali

Department of Artificial Intelligence
Cairo University

<2024><2nd Semester>

TABLE OF CONTENTS

List of figures 0
CHAPTER ONE: Introduction 1
1.1.Abstract 1
1.2.Problem definition 1
1.3.Model objectives and techniques 2
1.3.1. Speech-to-Text Conversion 2
1.3.2 .Text Editing 2
1.3.3 .Voice and Pitch Cloning 2
1.4.Model application 3
CHAPTER TWO: Related Work 3
2.1.Text-to-Speech(TTS) 3
2.2.Speech-to-Text (STT) 4
2.3.VITS 4
2.4.YourTTS 5
2.5.Tortoise 6
2.6.Bark 7
2.7.Voice Cloning 7
2.8.Distinctive features of this project 7
CHAPTER THREE: Project specifications 8
3.1 Dataset description 8
3.2 System input and output 8
3.3 Limitations 8
3.3.1. Sound Clarity Limitation 8
3.3.2. Increased Sound Variety 9
3.3.3. Audio Duration 9
CHAPTER FOUR: Model 9
4.1 Speech Recognition library 9
4.1.1. Benefits and functionalities 9
4.1.2. MAIN classes at the library 10
4.1.3. HOW IT WORKS 11
4.2 Arabic-reshaper 12
4.3 Python-bidi 12
4.4 XTTS model 13
4.4.1. Model architecture 14
4.4.2. HOW IT WORKS 24

4.5 Use Case Diagram 25
CHAPTER FIVE: Experimental Results 26
5.1 training performance metrics 26
5.1.1 loss_text_ce 26
5.1.2 loss_mel_ce 27
5.1.3 loss 28
5.1.4 current_lr 28
5.1.5 step_time 28
5.1.6 loader_time 28
5.2. Evaluation performance metrics 29
5.2.1 avg_loader_time 29
5.2.2 avg_loss_text_ce 29
5.2.3 avg_loss_mel_ce 29
5.2.4 avg_loss 29
5.3 Results and Analysis 29
6. Audify user interface 30
7. Conclusions and Future Work 32
7.1 Conclusions 37
7.2 Future Work 37
8.References 38

List of figures

Figure 1: System input and output _____ 8
Figure 2 : VAE architecture _____ 15
Figure 3 : Synthesizer _____ 18
Figure 4: GPT2 Model _____ 19
Figure 5: hif-gan model_____ 22
Figure 6: XTTS architecture_____ 24
Figure 7: Use case diagram _____ 25
Figure 8: Eval performance _____ 30
Figure 9: Training Performance _____ 30
Figure 10 - Figure 14: User interface _____ 32
Figure 15: Project Stages _____ 40
Figure 16,17: plane achievements _____ 41

CHAPTER ONE: Introduction

1.1.Abstract

In an era dominated by multimedia content, the significance of audio quality cannot be overlooked. In this project we are invested in the creation of a dynamic voice editing system. Named "Audify" this project aims to redefine the field of audio recording by providing users with an easy to use and versatile platform for editing voice records.

SIMPLYFING:

1.2.Problem definition

Due to human error, audio recordings can sometimes be challenging to produce perfectly over extended intervals. As a result, one or two words may be inadvertently omitted from the transcript or misinterpreted, causing the failure of over 200 words. In our project, the goal is to address these occasional errors without the need to delete the entire audio file. This is crucial as deleting the entire file is both tedious and time-consuming. Our project focuses on efficiently fixing these specific instances of missing words, aiming to provide a more targeted and practical solution.

1.3.Model objectives and techniques

The primary objective of the model is to rectify flaws present in an audio file under the user's guidance. The process involves employing straightforward techniques to enhance the overall quality of the audio content. The following steps outline the key techniques utilized

1.3.1. Speech-to-Text Conversion

The model begins by converting the user-provided audio into readable text using a speech-to-text model. This initial step aims to transcribe the spoken content, laying the groundwork for subsequent edits.

1.3.2 . Text Editing

After the speech-to-text conversion, the user is required to edit the transcribed text. This step enables the user to rectify any flaws detected in the initial transcription, ensuring the final output aligns with their intended message.

1.3.3 .Voice and Pitch Cloning

After the user completes the editing process, the rectified text is converted back into an audio file using a text-to-speech model. The final phase involves cloning the user's voice and pitch to generate a new audio file with the corrected flaws. By preserving the user's unique vocal characteristics, this step ensures a seamless output.

1.4.Model application

The model is designed to operate specifically on the Egyptian dialect of Arabic, with an intention to expand its functionality to include additional dialects and languages in the future.

1.5 Stakeholders

Our project doesn't specifically target any particular stakeholder, entity, or organization. Its primary objective is to be of general interest, allowing individuals, studios, companies, or any professional entity to easily benefit from it. Thus, it is designed for broad accessibility and usability.

CHAPTER TWO: RELATED WORK

2.1.Text-to-Speech (TTS)

There have been a lot of TTS models that convert speech to text such as:

- Tacotron 2 is a text-to-speech model with an encoder-decoder architecture, featuring an attention mechanism for improved performance. The model processes input text, generates mel spectrogram frames in the decoder, and uses a WaveNet-based vocoder for waveform synthesis. Trained on a large dataset, Tacotron 2 is capable of handling multiple languages and has been influential in advancing high-quality text-to-speech synthesis for various applications.

2.2.Speech-to-Text (STT)

- Silero Models is pre-trained enterprise-grade Speech-to-Text (STT) and Text-to-Speech (TTS) models, emphasizing simplicity and high-quality performance, comparable to Google's STT. Notably, Silero models do not require Kaldi, compilation, or complex instructions. The TTS models feature one-line usage, a diverse voice library, end-to-end pipeline, natural-sounding speech, and operate faster than real-time on a single CPU thread. Additionally, Silero provides a model for text repunctuation and recapitalization across four languages, offering improved readability without relying on hard-coded rules.

- Klaam Models is a software for Arabic speech recognition, classification and text-to-speech conversion using many advanced models such as wave2vec and fastspeech2. This model allows training and prediction using pre-trained models trained on the Egyptian dialect

2.3.VITS

The VITS (Variational Inference Text-to-Speech) model represents a significant advancement in the field of text-to-speech synthesis. VITS utilizes a combination of variational inference and adversarial training, resulting in more natural and expressive speech generation. The model leverages a variational autoencoder (VAE) to model the latent space of speech and a generative adversarial network (GAN) to enhance the realism of the generated audio. This dual approach allows VITS to produce high-quality speech that closely mimics human intonation and rhythm.

In contrast, XTTS (Cross-lingual Text-to-Speech) models are designed to handle the synthesis of speech in multiple languages, often focusing on the transfer of linguistic features across different languages.

XTTS models typically involve intricate processes to maintain linguistic nuances and pronunciation accuracy across languages. While both VITS and XTTS aim to improve the quality and naturalness of synthesized speech, VITS primarily focuses on leveraging advanced generative techniques to enhance audio realism, whereas XTTS emphasizes multilingual capabilities and the preservation of linguistic characteristics across different languages.

By integrating variational inference with adversarial training, VITS achieves superior speech quality and expressiveness, setting it apart from traditional TTS models and offering a new benchmark for natural-sounding speech synthesis.

2.4.YourTTS

The YourTTS model is an innovative text-to-speech synthesis system that stands out due to its ability to generate personalized and multilingual speech. YourTTS leverages the combination of voice cloning and multilingual capabilities, enabling it to produce speech in various languages while retaining the unique characteristics of an individual's voice. This is achieved through the use of advanced neural network architectures and fine-tuning techniques that allow the model to adapt to different languages and accents seamlessly.

In contrast, XTTS (Cross-lingual Text-to-Speech) models are primarily focused on synthesizing speech across multiple languages, emphasizing the accurate transfer of linguistic features and maintaining pronunciation fidelity. While XTTS models excel in producing high-quality speech in different languages, they do not typically offer the level of personalization seen in YourTTS. The primary distinction lies in YourTTS's ability to clone specific voices and generate personalized speech across various languages, whereas XTTS models focus more on the linguistic and phonetic accuracy across language boundaries.

Overall, YourTTS's unique approach to combining voice cloning with multilingual text-to-speech synthesis sets it apart from XTTS models, offering a more personalized and versatile solution for generating natural and expressive speech in multiple languages

2.5.Tortoise

The Tortoise model represents a significant advancement in the field of text-to-speech (TTS) synthesis, offering improvements over traditional TTS systems such as XTTS. The Tortoise model leverages state-of-the-art deep learning techniques to produce more natural and expressive speech. Unlike XTTS, which typically relies on concatenative or parametric methods, the Tortoise model utilizes a transformer-based architecture. This allows it to generate high-fidelity audio with better prosody and intonation, capturing the nuances of human speech more effectively. Additionally, the Tortoise model incorporates advanced neural network components that enable it to handle a wider range of voices and accents, providing a more versatile and scalable solution for various applications. This capability is particularly beneficial in scenarios requiring high-quality and diverse speech outputs, such as virtual assistants, audiobooks, and multilingual communication systems. By addressing the limitations of XTTS, the Tortoise model represents a significant step forward in the quest for more realistic and engaging synthetic speech.

2.6.Bark

The Bark model is an innovative text-to-speech (TTS) system that stands out due to its advanced neural network architecture and natural language processing capabilities. Unlike XTTS, which relies on traditional concatenative synthesis or parametric models, Bark employs deep learning techniques to generate high-quality, natural-sounding speech. This allows Bark to produce more expressive and nuanced vocal outputs, closely mimicking human intonation and rhythm. Additionally, Bark's robust training on diverse datasets enables it to handle a wide range of accents and languages with greater accuracy. In contrast, XTTS often struggles with the variability in speech patterns and may require extensive manual tuning to achieve similar results. Consequently, Bark offers a more scalable and adaptable solution for modern TTS applications, making it a superior choice for dynamic and interactive voice-based systems.

2.7.Voice Cloning

In recent times, numerous open-source voice cloning models have emerged, including Real-Time Voice Cloning (RVC) developed by Baidu. RVC leverages advanced deep neural networks to create natural-sounding voices, combining autoencoders and neural networks to capture and reproduce individual voice traits in real-time. This technology involves training the model on the target speaker's voice recordings, extracting voice features, and synthesizing speech that closely resembles the original, resulting in a lifelike synthetic version of a person's voice.

2.8.Distinctive features of this project

The primary distinction of our project lies in its integration of all three features: Text-to-Speech (TTS), Speech-to-Text (STT), and voice cloning, resulting in a comprehensive audio editing framework focused on spoken audio manipulation. Furthermore, our initiative has brought attention to the Egyptian dialect of Arabic, as it is often overlooked in TTS and STT models, particularly within open-source ones.

CHAPTER THREE: Project specifications

3.1 Dataset description

The dataset utilized in the model comprises an extensive compilation of Egyptian Arabic audio recordings paired with their corresponding transcriptions, capturing the essence of conversational speech in Egyptian Arabic. Each audio clip varies in length from 1 to 10 seconds, with a consistent sample rate of 22050 Hz. This meticulously curated dataset was gathered from the following resources: Kaggle, YouTube, Hugging Face, and Magic Hub, ensuring a diverse and comprehensive representation of spoken Egyptian Arabic.

Additionally, the majority of the dataset has been manually created by recording small clips, totaling 2 hours of content. These recordings feature a wide range of topics, letters, and unique words to enhance training diversity. This approach ensures a comprehensive coverage of different sounds and helps in recognizing letters and improving the model's ability to understand and generate conversational speech in Egyptian Arabic.

3.2 System input and output

The system takes an audio file as its initial input, and return the transcription of the audio to the client to be able to edit it, then takes the edited text an input to produce a new audio file identical to what the client modified with voice cloning

Figure 1

3.3 Limitations

3.3.1. Sound Clarity Limitation

The success of the models depends on the clarity of the sound in the audio data, whether it is the audio used for training the voice cloning model or the audio passed to the audio transcription model. If the input audio has poor quality or unclear speech, it may affect the models' results.

3.3.2. Increased Sound Variety

Enhancing the quality of voice cloning is achievable by providing a diverse array of sounds during training. Expanding the range of sounds in the (speaker_wav) list has the potential to enrich the model's capacity to capture various characteristics in the speaker's voice making it sound more like how the end user speaks.

3.3.3. Audio Duration

The audio passed to the audio transcription model cannot exceed 1 minute duration. And for better results with the voice cloning model, it's recommended to use audio data with a duration not less than two minutes for training. Shorter durations may limit the model's ability to comprehensively learn the speaker's voice characteristics

---

CHAPTER FOUR: Model

4.1 SpeechRecognition library

SpeechRecognition is a library that provides easy-to-use functions for performing speech recognition, converting spoken language into text. Functionality: SpeechRecognition allows you to recognize speech from various sources, including audio files and microphone input, and convert it into text

4.1.1. Benefits and functionalities

- Support for Multiple APIs, including Google Web Speech API, CMU Sphinx, Microsoft Bing Voice Recognition, IBM Speech to Text, and more.
- This flexibility allows you to choose the best API based on your needs.
- Easy Integration
- Installing via pip (the python package).

4.1.2. MAIN classes at the library

THE LIBRARY contains classes and functions that facilitate the process of converting speech to text using various APIs and engines.

1. **Recognizer Class**: This is the core class in SpeechRecognition. It provides methods for recognizing speech from audio sources such as audio files and microphone inputs. Some key methods of the Recognizer class include:

- recognize_google(): Recognizes speech using the Google Web Speech API.
- recognize_bing(): Recognizes speech using Microsoft Bing Voice Recognition API.
- recognize_sphinx(): Recognizes speech using CMU Sphinx.
- recognize_wit(): Recognizes speech using the Wit.ai API.
- recognize_ibm(): Recognizes speech using IBM Speech to Text service.

2. **AudioFile Class**: This class represents an audio file. You can use it to load audio files and perform speech recognition on them. It provides **methods** like **record()** for recording from a microphone and **recognize()** for recognizing speech from the loaded audio file.

3. **Microphone Class**: This class provides access to the microphone for real-time speech recognition. It has methods for recording audio from the microphone and converting it to text.

4. **AudioData Class**: This class represents audio data. It is used to store audio data and metadata obtained from audio sources such as files and microphones. It provides methods for converting audio data to text

4.1.3. HOW IT WORKS

1. **Input Audio**: The first step is to provide input audio containing spoken language. This audio can come from various sources, such as audio files (e.g., WAV, MP3) or real-time microphone input.

2. **Recognizer Initialization**: You create an instance of the Recognizer class provided by the SpeechRecognition library. This class provides methods to recognize speech from different sources using various APIs and engines.

3. **Audio Source**: Depending on the source of the input audio, you may need to use different classes provided by SpeechRecognition. For example:

- If you're working with an audio file, you can use the AudioFile class to load the file.
- If you want to perform real-time recognition from a microphone, you can use the Microphone class to capture audio.

4. **Speech Recognition**: Once you have the audio data, you can use the Recognizer instance's methods to recognize speech. You choose the appropriate method based on the API or engine you want to use for recognition (e.g., Google Web Speech API, CMU Sphinx, etc.).

5. **Output Text**: After the recognition process, the Recognizer method returns the recognized text. This text represents the spoken language in the input audio.

6. **Error Handling**: During the recognition process, errors might occur due to various reasons, such as network issues or inability to recognize speech accurately. SpeechRecognition provides exception handling mechanisms to deal with these errors gracefully.

4.2 Arabic-reshaper

This library primarily focuses on reshaping Arabic text to ensure proper rendering in various contexts.

Purpose:

- The Arabic-Reshaper library addresses two essential features of Arabic script:
  - Right-to-left writing: Arabic text is written from right to left.
  - Dynamic character shaping: Characters change shape based on their surrounding characters.

Functionality:

- When you print Arabic text in an application or library that lacks native Arabic support, you often end up with isolated characters and left-to-right rendering.
- The library reshapes the characters, ensuring they appear correctly based on their context.

4.3 Python-bidi

simplifies bidirectional text handling, making it an essential tool for applications dealing with mixed-directional content.

Purpose:

- The primary purpose of python-bidi is to handle the complexities of bidirectional text rendering.
- It ensures that text containing both left-to-right (LTR) and right-to-left (RTL) characters displays correctly.

4.4 XTTS Model

The XTTSv2 model is an advanced Text-to-Speech (TTS) generation model developed by Coqui TTS. It is designed to revolutionize voice generation by enabling voice cloning across languages with minimal audio input. Unlike traditional TTS models that require extensive training data spanning countless hours, XTTSv2 achieves voice cloning efficiently.

Here are its key features:

- Language Support: XTTSv2 currently supports 17 languages, including English, Spanish, French, German, Italian, Portuguese, Polish, Turkish, Russian, Dutch, Czech, Arabic, Chinese, Japanese, Hungarian, Korean, and Hindi. The team continues to add support for more languages.
- Emotion and Style Transfer: XTTSv2 allows for emotion and style transfer by cloning. Whether you want to convey warmth, excitement, or seriousness, this model adapts to your desired tone.
- Multi-lingual Speech Generation: XTTSv2 seamlessly generates speech in multiple languages. It's a powerful tool for applications that require multilingual voice output.

4.4.1. Model Architecture

The model consists of three main parts: the speaker encoder, synthesizer, and vocoder.

A. the speaker encoder (VAE)

I.Introduction

The speaker encoder integrates a Discrete VAE (Variational Autoencoder) model and a Perceiver model. The Discrete VAE specializes in compressing high-dimensional audio signals into discrete audio tokens, which provide a more manageable representation for the GPT2 model to process. These tokens serve as a compact representation of the audio data. Meanwhile, the Perceiver model, adept at handling complex data, processes audio inputs in the form of mel spectrograms and outputs 32 latent vectors that capture the speaker's voice characteristics. These vectors are then utilized to condition the GPT2 model for more nuanced responses and interactions. Additionally, a VQ-VAE (Vector Quantised-Variational AutoEncoder) with 13M parameters processes mel-spectrogram inputs, using a codebook with 8192 codes per frame at a 21.53 Hz rate. Following training, the codebook is filtered to retain only the top 1024 most frequent codes, enhancing the model's expressive capabilities based on initial experimental findings.

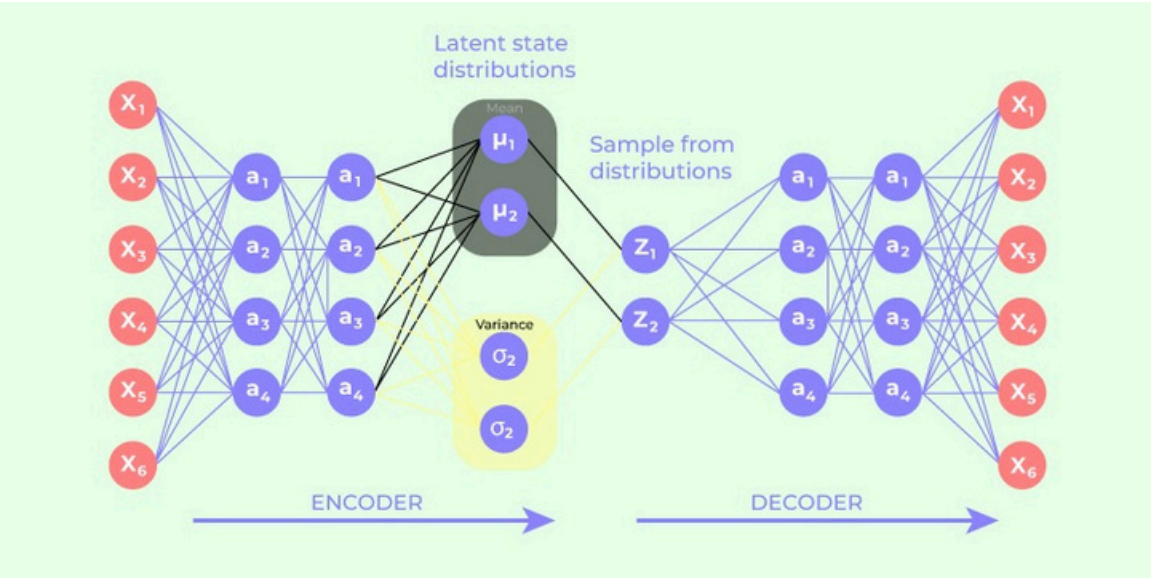

Figure 2

II.What is a Variational Autoencoder (VAE)?

A variational autoencoder is a type of generative neural network architecture. At its heart, a VAE still has the same structural components as a traditional autoencoder: an encoder and a decoder.

- Encoder: The encoder's job is to take an input (like an image) and compress it into a compact, latent representation. Traditional autoencoders output a single point in the latent space. A VAE, however, outputs a distribution (usually a Gaussian) over the latent space.

- Decoder: The decoder samples a point from this distribution in the latent space and attempts to reconstruct the original input as accurately as possible.

III.Why Use a VAE?

Generative Power: The key advantage of VAEs lies in their generative capabilities. Because a VAE learns to model the underlying probability distribution of the input data, it can **generate new samples** that are similar to the original data. Think generating realistic faces, composing new pieces of music, or crafting creative text.

Continuous Latent Space: Unlike traditional autoencoders, VAEs promote a continuous and smooth latent space. This means that we can interpolate between points in this space to generate smoothly transitioning variations of our data. Imagine smoothly morphing one person's face into another's!

IV.VAE vs. Autoencoder: Differences

Variational Autoencoder (VAE)
Latent Space: Continuous and smooth, enabling interpolation and meaningful transitions of representations.
Generative Power: Powerful generative capabilities to produce new, similar data samples.
Loss: Reconstruction loss combined with KL divergence to regularize the latent space, forcing it to follow a prior distribution (often Gaussian).

Autoencoder (AE)
·**Latent Space:** Potentially discrete, which can lead to gaps or abrupt changes in representations.
·**Generative Power:** More limited in generating entirely new data samples.
Loss: Primarily focused on reconstruction loss, ensuring accurate reproduction of the input.

V.How Does a VAE Work?

1. The Distribution Trick: Instead of a point in the latent space, the encoder of a VAE outputs the parameters that define a probability distribution (usually mean and variance). During training, we sample a point from this distribution to feed into the decoder.

2. The Reparameterization Trick: This is the clever part. Directly backpropagating gradients through random sampling is tricky. The reparameterization trick lets us express the sampled point in the latent space as a deterministic function of the distribution parameters and an external random variable. This allows for proper training.
3. Loss Function: Beyond Reconstruction: The VAE's loss function has two parts:

- **Reconstruction Loss**: Just like in an autoencoder, this part ensures that the decoder accurately reconstructs the input.

- **KL Divergence**: This is where the probabilistic twist comes in. The Kullback-Leibler (KL) divergence measures how much the encoder's learned distribution diverges from a standard prior distribution (often a standard Gaussian). This encourages a well-structured, regular latent space.

B. Synthesizer

I.Introduction

The synthesizer consists of the GPT2 model, a language model provided by OpenAI. It takes the discrete tokens output by the Discrete VAE model and the 32 latent vectors output by the perceiver model. These inputs allow the GPT2 model to generate audio tokens that not only follow linguistic patterns in the input text but also mimic specific voice characteristics of the speaker.

II.What is Synthesizer

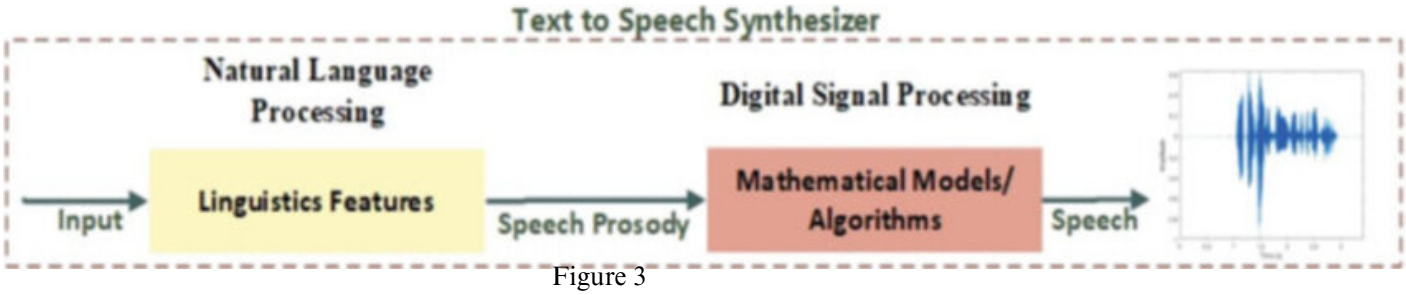

Figure 3

A synthesizer, in the context of modern AI, is a system designed to generate audio, often mimicking human speech or other sound patterns. The process typically involves several key components working together to transform input data into coherent audio output. In this specific setup, the synthesizer leverages the capabilities of various models, including the GPT-2 language model, the Discrete VAE (Variational Autoencoder), and the Perceiver model.
The Discrete VAE model first processes audio input into discrete tokens, capturing essential audio features in a compressed form.

These tokens serve as a foundational representation of the sound's characteristics. Simultaneously, the Perceiver model generates 32 latent vectors that encapsulate additional context and nuances of the audio or speaker's voice.
These inputs are then fed into the GPT-2 model, a powerful language model developed by OpenAI. The GPT-2 model processes the discrete tokens and latent vectors to generate new sequences of audio tokens. These tokens are designed to follow the linguistic patterns presented in the input text while also incorporating specific voice characteristics, resulting in audio that not only makes sense linguistically but also sounds like the intended speaker.

III.GPT-2

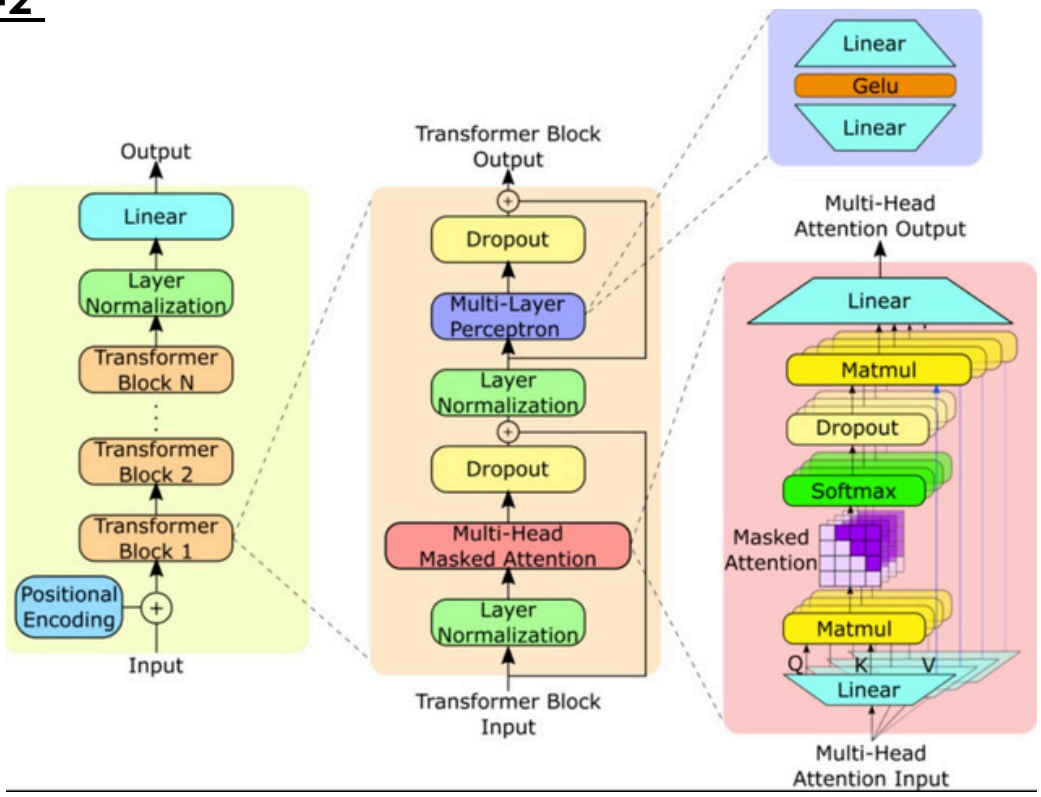

Figure 4

The GPT-2 (Generative Pre-trained Transformer 2) model is an advanced language model developed by OpenAI. It is based on the Transformer architecture, which has proven exceptionally effective in understanding and generating human-like text. GPT-2 is pre-trained on a diverse and extensive corpus of text data, enabling it to generate coherent and contextually relevant text based on the input it receives.

The model works by predicting the next word in a sequence, given all the previous words. This ability to predict and generate text sequentially allows GPT-2 to create paragraphs, articles, and even dialogue that flow naturally. During training, the model learns the statistical properties of language, such as grammar, vocabulary, and even some level of contextual meaning. When fine-tuned for specific tasks, GPT-2 can adapt to generate text that fits particular styles or domains.
In the context of the synthesizer, GPT-2 extends its text generation capabilities to audio token generation. By integrating the discrete tokens from the VAE model and the latent vectors from the Perceiver model, GPT-2 can produce audio tokens that accurately reflect both the linguistic structure of the input text and the voice characteristics of the speaker, resulting in highly realistic and personalized audio outputs.

C. Vocoder

C.1 Vocoder

I.Introduction

The vocoder consists of the HifiGAN model, a type of Generative Adversarial Network (GAN) specifically designed for high-fidelity speech synthesis. It receives the predicted audio tokens from the GPT2 model, representing linguistic and speaker-specific information for synthesized speech, and converts them into a highresolution waveform, the final audio signal.

II.What is a Vocoder?

A vocoder (short for "voice encoder") is an audio processing tool that analyzes and synthesizes the human voice signal. It can transform voice signals into a different form and then reconstruct them, often used in speech synthesis, music production, and telecommunications.

III.How Does a Vocoder Work?

A traditional vocoder works by:

1.Analysis: The input voice signal is analyzed to extract its spectral characteristics.

2.Modulation: These characteristics are then used to modulate another signal, often called the carrier signal.

3.Synthesis: Finally, the modulated signal is combined with the carrier to produce the output.

In modern applications, vocoders are used in text-to-speech systems to convert linguistic and speaker-specific information into audio signals.

C.2 HiFi-GAN Model

I.What is HiFi-GAN?

HiFi-GAN (High-Fidelity Generative Adversarial Network) is a type of GAN specifically designed for high-fidelity audio synthesis, particularly in text-to-speech systems. It aims to generate realistic and high-quality audio waveforms from intermediate representations like mel-spectrograms or other features derived from text.

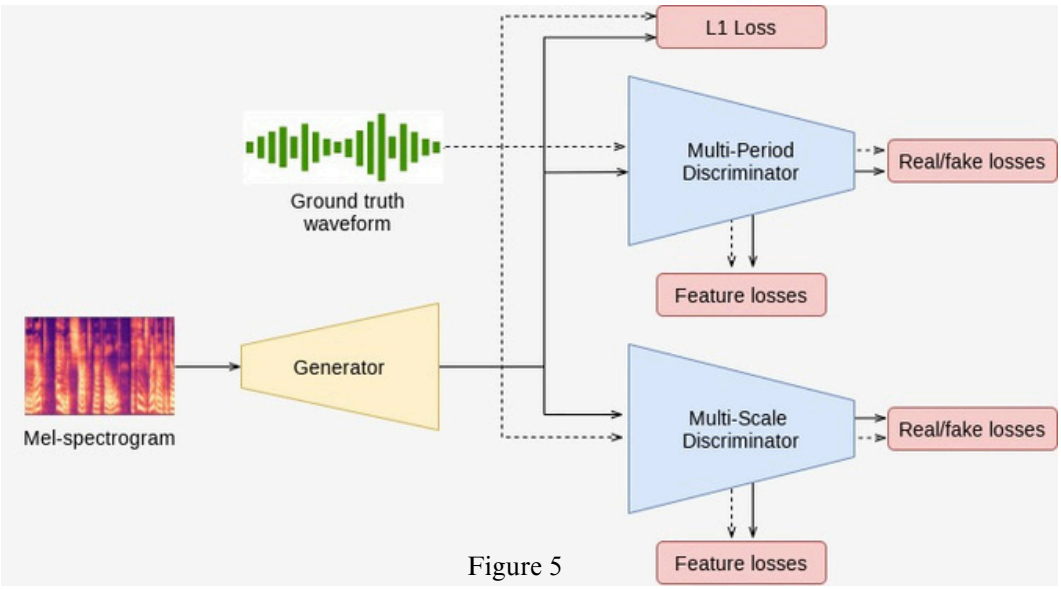

Figure 5

II.How Does HiFi-GAN Work?

HiFi-GAN uses a GAN architecture, which consists of two neural networks:

1.Generator: Converts the input features (e.g., mel-spectrograms) into a high-resolution audio waveform.

2.Discriminator: Evaluates the quality of the generated audio by distinguishing between real and synthesized audio.

The two networks are trained together in a competitive manner:

- The **generator** tries to create audio that is indistinguishable from real audio.
- The **discriminator** tries to correctly identify whether the audio is real or generated.

Over time, this adversarial process helps the generator improve its ability to produce high-quality, realistic audio.

C.3 Generative Adversarial Network (GAN)

I.What is a GAN?

A GAN is a class of machine learning frameworks designed for generative tasks. It involves two neural networks, the generator and the discriminator, which play a min-max game during training.

Components of a GAN

1.Generator: Takes random noise (or some other input) and generates data that resembles the real data distribution.

2.Discriminator: Evaluates the authenticity of the data, distinguishing between real data (from the training set) and fake data (produced by the generator).

II.How Does a GAN Work?

The training process of a GAN involves the following steps:

1.Generator Training: The generator produces fake data samples from random noise.

2.Discriminator Training: The discriminator evaluates the fake samples and real samples, providing feedback on whether they are real or fake.

3.Adversarial Training: The generator is updated based on the discriminator's feedback to produce more realistic samples. Simultaneously, the discriminator is updated to better distinguish real from fake samples.

This iterative process continues until the generator produces high-quality data that the discriminator can no longer differentiate from real data.

4.4.2 HOW IT WORKS

The user is required to provide two inputs, the audio recordings in their voice along with the text to be cloned in their voice. These recordings are passed to the speaker encoder to capture the speaker's embedding and the audio tokens, which are then passed to the synthesizer. The synthesizer takes these inputs along with the text to be cloned in the user's voice and predicts the audio tokens for the given text in the user's voice. Finally, the predicted audio tokens are passed through the vocoder to generate the final audio signal (audio waveform) that sounds like the user's voice speaking the input text.
This process allows the XTTSv2 model to clone the user's voice and generate audio of the user speaking the input text.

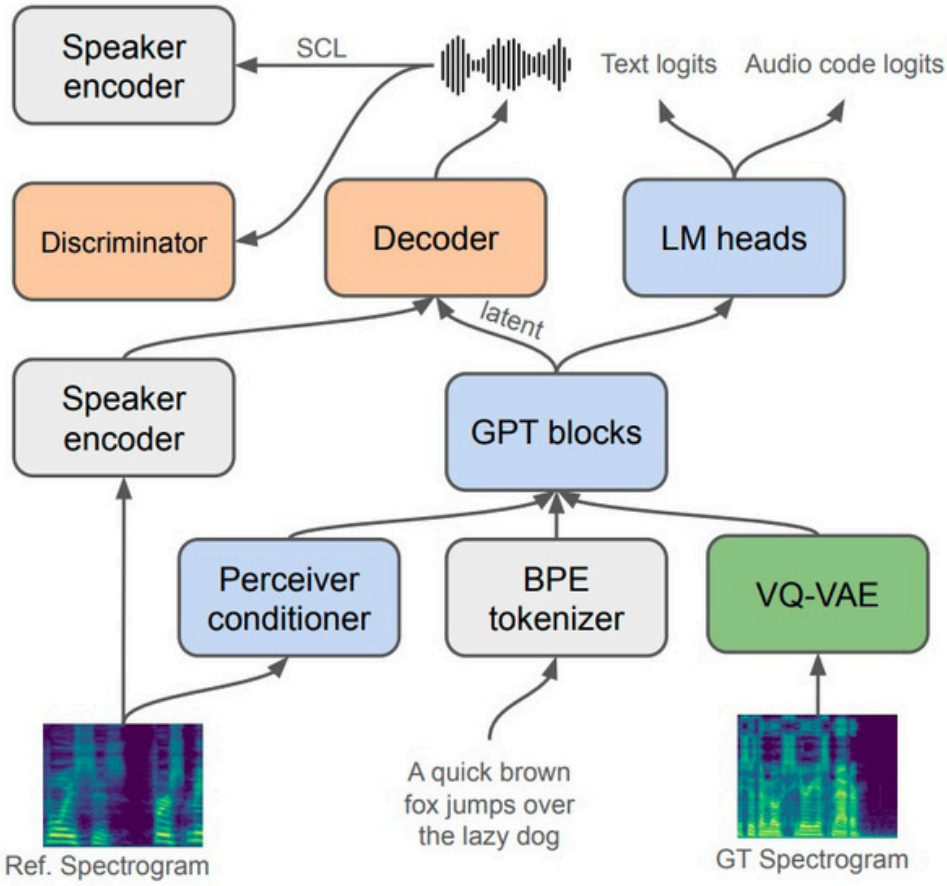

Figure 6

4.5 USE CASE DIAGRAM

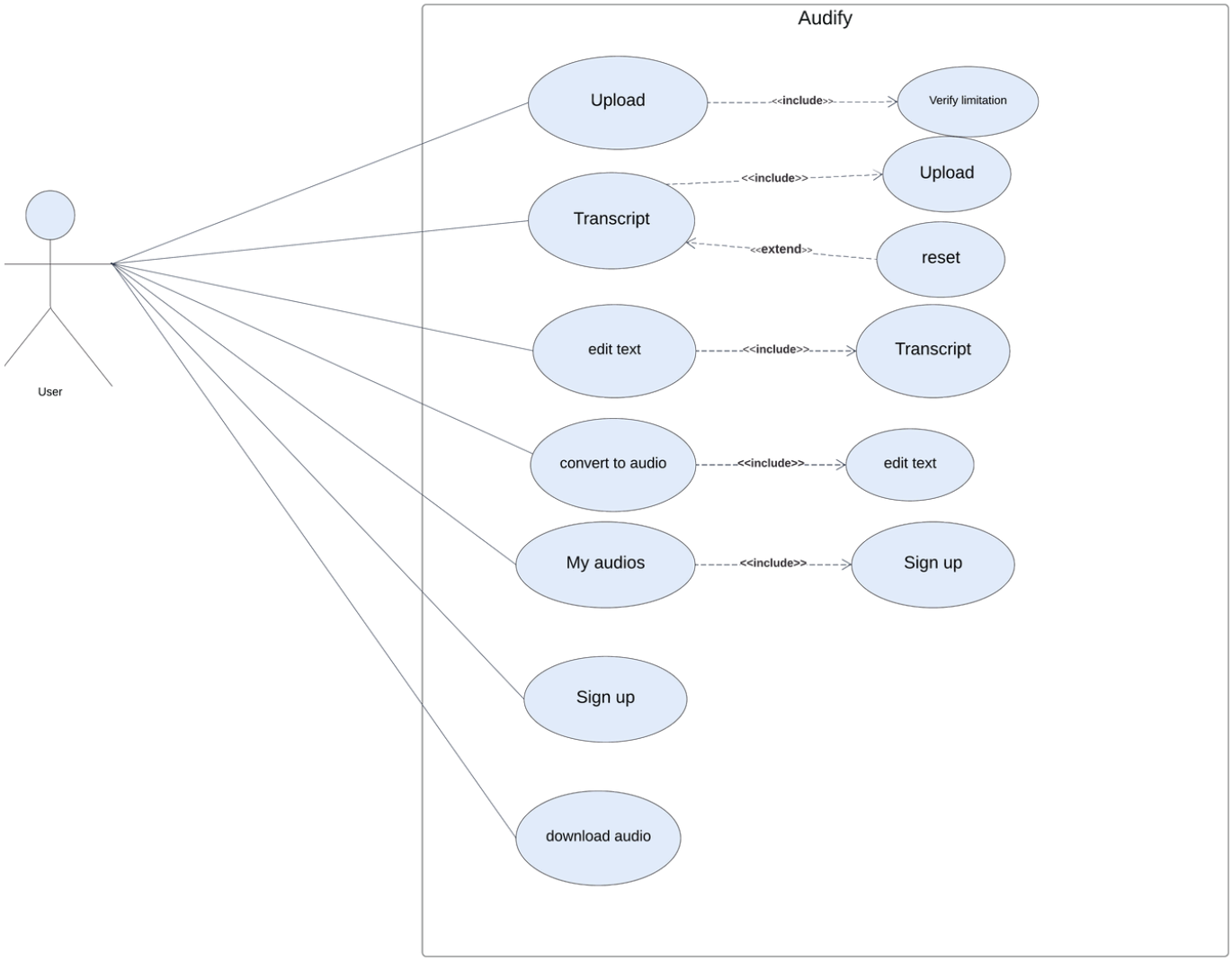

Figure 7

CHAPTER FIVE: Metrics

5.1 training performance metrics

Training performance measures are essential tools for evaluating the effectiveness of training programs. In the context of AI, these measures can assess how well AI models and algorithms achieve training objectives

5.1.1 loss_text_ce:

represents the cross-entropy loss for the text prediction component of the model. This metric measures how well the model predicts the correct text from the input Lower values for loss_text_ce indicate better model performance. Monitoring these values over time ensures that the model is learning and improving. Loss metrics, including cross-entropy loss, are computed during the forward pass of the network. Cross-entropy loss specifically measures the difference between the predicted probability distribution and the true distribution.

Equation:

$$\text{loss_text_ce} = -\sum_{i=1}^{N} \sum_{j=1}^{C} y_{ij} \log(\hat{y}_{ij})$$

where:

- N is the number of samples.
- C is the number of classes (e.g., phonemes or characters).
- yij is the true probability distribution of the j-th class for the i-th sample (one-hot encoded).
- y^ij is the predicted probability distribution of the j-th class for the i-th sample.

If the model predicts the correct text with high probability, the cross-entropy loss will be low.

5.1.2 loss_mel_ce:

Represents the cross-entropy loss for the mel-spectrogram prediction component of the model. This metric measures how well the model predicts the correct mel-spectrogram from the input text Lower values and overall loss indicate better model performance. Monitoring these values over time ensures that the model is learning and improving. Accurate mel-spectrogram prediction is essential for generating high-quality, natural-sounding speech, making this loss crucial for ensuring that the text-to-speech model correctly understands and processes the input text before converting it into speech. This component directly affects the quality of the generated audio, emphasizing its importance in the actual synthesis of speech.

Equation:

$$\text{loss_mel_ce} = -\sum_{i=1}^{N}\sum_{t=1}^{T}\sum_{m=1}^{M} y_{itm}\log(\hat{y}_{itm})$$

where:

- N is the number of samples.
- T is the number of time frames in the mel-spectrogram.
- M is the number of mel-spectrogram bins.
- y_{itm} is the true probability distribution for the m-th mel-spectrogram bin at time t for the i-th sample (one-hot encoded).
- $\hat{y}_{itm}$ is the predicted probability distribution for the m-th mel-spectrogram bin at time t for the i-th sample.

5.1.3 loss:

Represents the overall loss, which combines the text prediction loss and the mel-spectrogram prediction loss. This metric provides a holistic measure of the model's performance, accounting for both the accuracy of text prediction and the quality of mel-spectrogram prediction. The overall loss should decrease as both text prediction and mel-spectrogram prediction improve. By tuning the weighting factors a and β, you can emphasize one component over the other based on the desired performance characteristics of your text-to-speech (TTS) model.

Equation:

$$\text{loss} = \alpha \cdot \text{loss_text_ce} + \beta \cdot \text{loss_mel_ce}$$

where:

- α and β are weighting factors that balance the contributions of text prediction loss and mel-spectrogram prediction loss.

5.1.4 current_lr:

indicates the current learning rate being used during the training process, which helps you understand the pace at which the model is learning. A high learning rate might cause the model to converge too quickly to a sub-optimal solution, while a low learning rate might make the training process unnecessarily long.

5.1.5 step_time:

measures the time taken for a single training step

5.1.6 loader_time :

measures the time taken to load the data for a single training step. These metrics are crucial for understanding the efficiency of the training process; lower values indicate faster training, which is desirable for reducing overall training time. The learning rate is set by the learning rate scheduler or optimizer configuration, while the timing metrics are measured using timing functions that track the duration of each training step and data loading.

5.2. Evaluation performance metrics:

quantitative measures used to assess the effectiveness of a model in achieving its intended tasks. These metrics provide insights into various aspects of the model's performance, such as accuracy, precision, recall, and overall efficiency. By analyzing these metrics, developers and researchers can identify strengths and weaknesses in the model, guide improvements, and ensure that the model meets the desired performance standards.

5.2.1 avg_loader_time:

This metric represents the average time taken to load each batch of data during evaluation. Lower values indicate more efficient data loading.

5.2.2 avg_loss_text_ce:

• This is the average cross-entropy loss for the text component of the TTS model.
• : Lower values indicate better performance of the text prediction part of the model.

5.2.3 avg_loss_mel_ce:

• This is the average cross-entropy loss for the mel-spectrogram predictions.
• Lower values indicate better performance of the audio (mel-spectrogram) prediction part of the model.

5.2.4 avg_loss:

This represents the average total loss, which is a combination of the text and mel-spectrogram losses.
• Lower values indicate better overall performance of the model

5.3 Results and Analysis

```
--> EVAL PERFORMANCE
   | > avg_loader_time: 0.04554259777069092 (-0.001217961311340332)
   | > avg_loss_text_ce: 0.03507043048739433 (-6.910413503646851e-06)
   | > avg_loss_mel_ce: 4.222620487213135 (+0.0030748844146728516)
   | > avg_loss: 4.257691144943237 (+0.003068208694458008)
```

Figure 8

```
--> TIME: 2024-06-27 20:32:06 -- STEP: 54/77 -- GLOBAL_STEP: 11450
  | > loss_text_ce: 0.035315997898578644  (0.03441440924588178)
  | > loss_mel_ce: 2.409688711166382  (2.515729743021506)
  | > loss: 0.029107199981808662  (0.030358859475840022)
  | > current_lr: 5e-06
  | > step_time: 0.3251  (0.3599230845769246)
  | > loader_time: 0.0081  (0.008185271863584165)
```

Figure 9

The loader time is significantly higher during evaluation compared to training. This might be due to different data loading mechanisms or batch sizes during evaluation. Loss Text CE (Cross-Entropy) The loss for text cross-entropy is quite similar in both evaluation and training, indicating consistent performance. The slight decrease during evaluation suggests minor improvements. Loss Mel CE (Cross-Entropy) There is a significant difference in the mel loss between evaluation and training.

The value during training is much lower, possibly indicating over-fitting or different scales of loss during these phases. Total Loss The total loss during evaluation is much higher than during training. This might be due to the different components contributing to the total loss, with the mel loss being particularly high during evaluation. Current Learning Rate The learning rate is provided only in the training step context, which is 5e-06. This information is not available in the evaluation metrics. Step Time The step time is only provided for the training step, indicating the time taken per step during training. Conclusion Loader Time: Higher during evaluation. Loss Text CE: Consistent across evaluation and training. Loss Mel CE: Significantly higher during evaluation, indicating potential scaling or overfitting issues. Total Loss: Higher during evaluation due to the high mel loss. Learning Rate: Provided only for training. Step Time: Provided only for training, showing the per-step duration. These values indicate that while the text cross-entropy loss remains stable, the mel loss and total loss show significant discrepancies between training and evaluation, which could point to areas needing further investigation or refinement in the model.

6. AUDIFY USER INTERFACE

Home Page

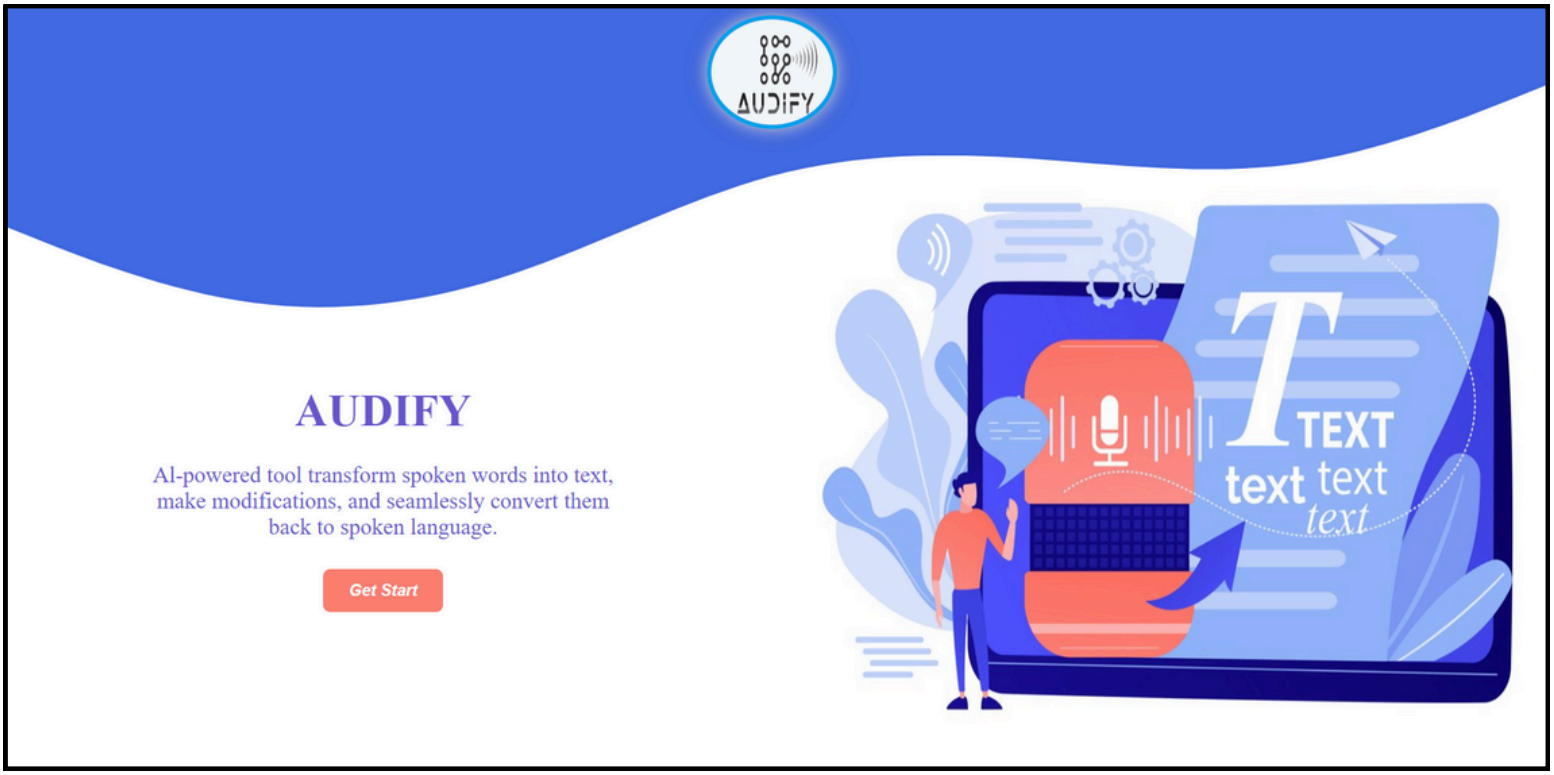

Figure 10

Upload Reference Audios

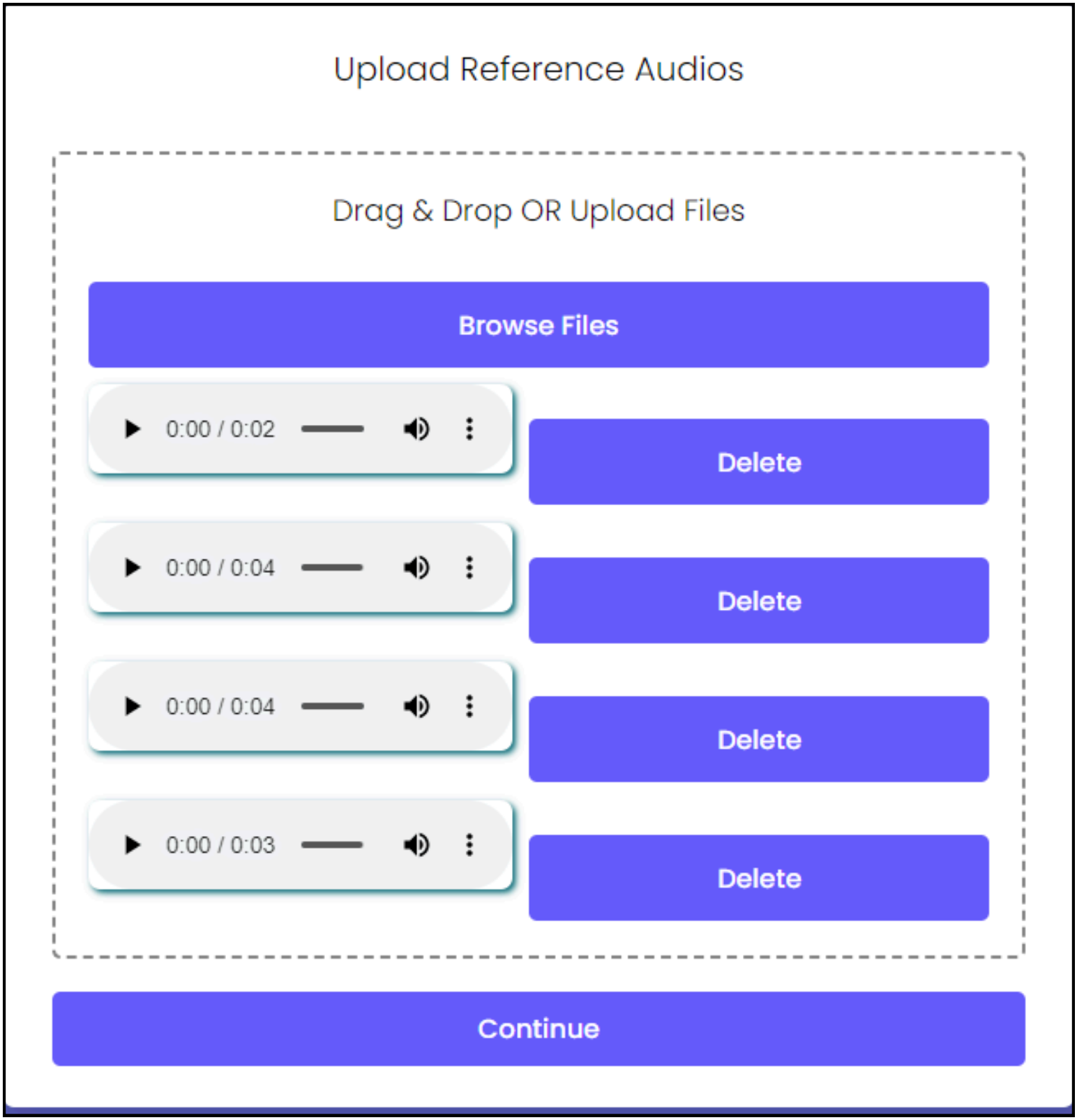

Figure 11

Edit page_1

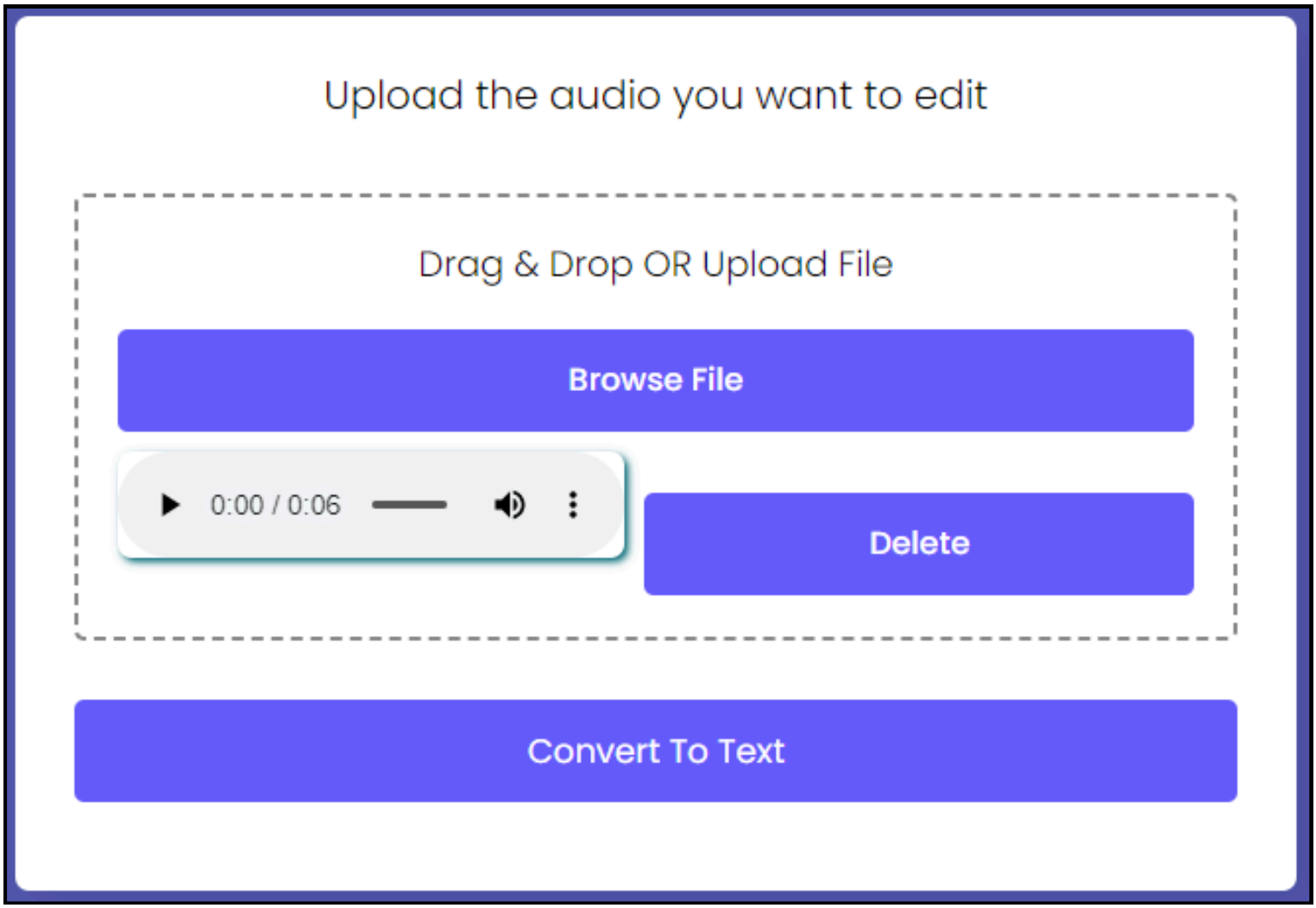

Figure 12

Edit page_2

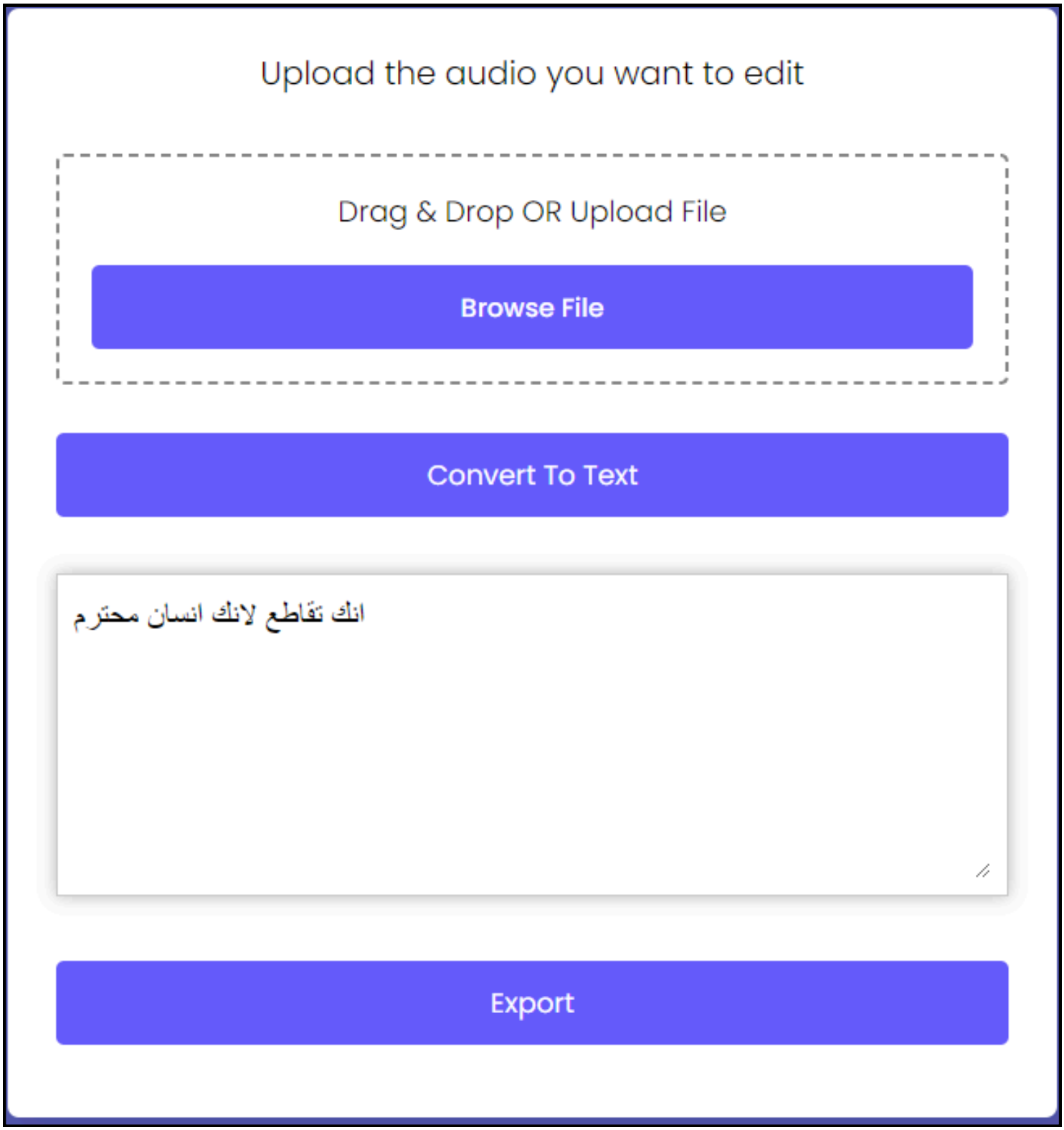

Figure 13

Edit page_3

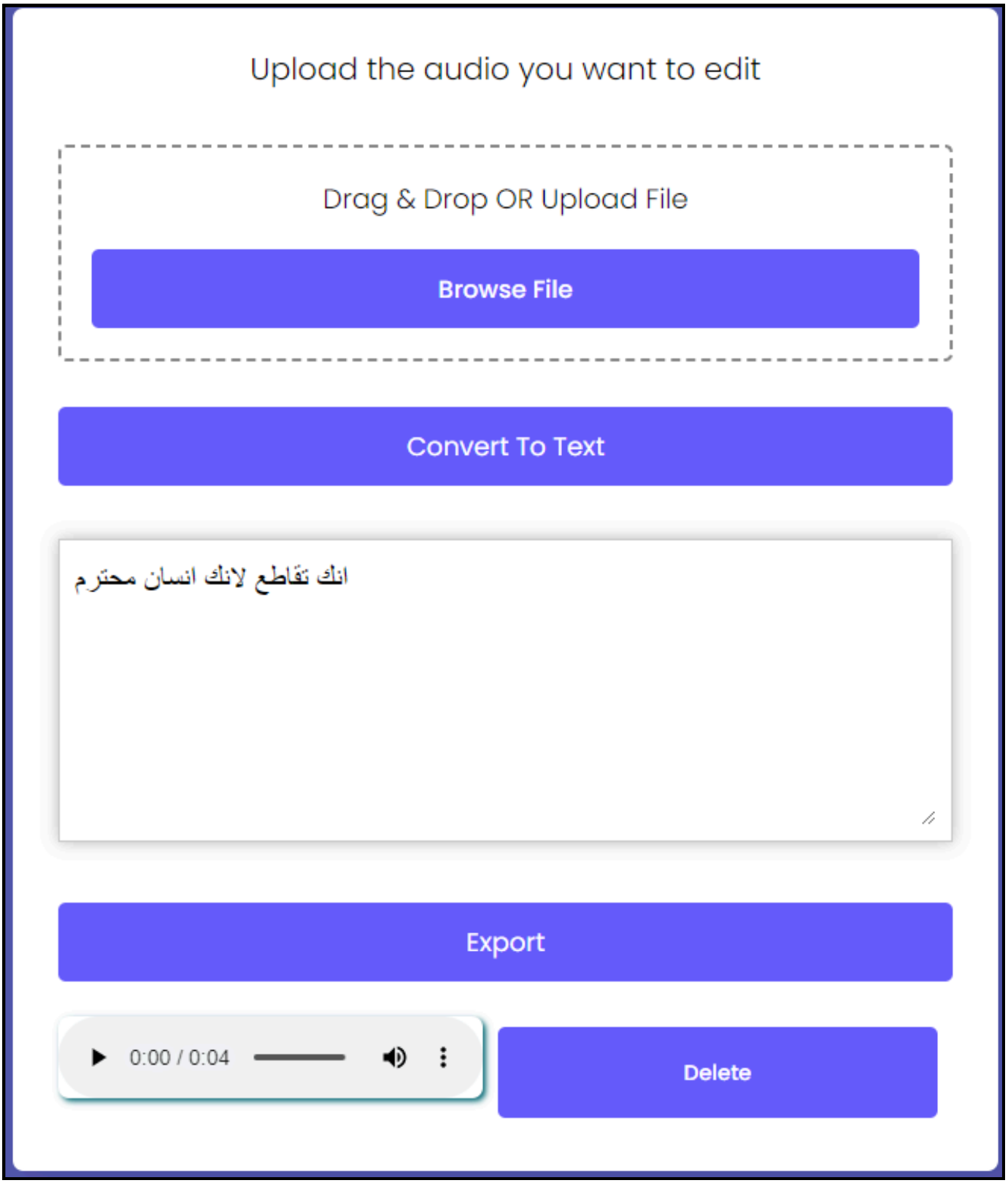

Figure 14

7. Conclusions and Future Work

7.1 Conclusions

Currently, our project is the first of its kind to combine text-to-speech and speech-to-text with voice cloning in the Egyptian language. We have integrated two models into our system. First, we use a speech recognition model, which is a library designed to convert spoken language to text. It provides functions to recognize speech from different sources, such as audio files and microphone inputs, and then convert this speech to text. Next, we use the XTTS v2 model to convert text to speech with voice cloning. XTTS v2 is a sophisticated model that provides high-quality voice synthesis and supports accurate voice cloning, making it ideal for the Egyptian dialect. We have carefully selected these models to be in line with our concept, as they strongly support the idea and are well suited for working with the Egyptian dialect. In addition, these models offer several other advantages, including the accuracy of voice cloning

7.2 Future Work

Enhance the Egyptian Dialect: We aim to further refine the Egyptian dialect to perfectly match the original speaker's tone by increasing the size of the data.
Work with Larger Audio Files: We will apply our system to larger audio files, including those on platforms like YouTube.
Integrate with Social Media: We plan to update our system to be compatible with social media applications, making it easy to edit and correct audio content on these platforms.
Extend to Video Editing: We will enable the project to work with videos as well, not just audio, allowing for comprehensive media editing capabilities.

8.REFERENCES

- FastSpeech 2: Fast and High-Quality End-to-End Text to Speech
  https://arxiv.org/pdf/2006.04558.pdf
- On decoder-only architecture for speech-to-text and large language model integration
  https://arxiv.org/pdf/2307.03917.pdf
- PERSONALIZED LIGHTWEIGHT TEXT-TO-SPEECH: VOICE CLONING WITH ADAPTIVE STRUCTURED PRUNING
  https://arxiv.org/pdf/2303.11816.pdf
- ArTST: Arabic Text and Speech Transformer
  https://aclanthology.org/2023.arabicnlp-1.5.pdf
- Advancements in Arabic Text-to-Speech Systems
  https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10078891
- ARABIC TEXT-TO-SPEECH (TTS) DATA PREPARATION
  https://arxiv.org/pdf/2204.03255.pdf
- Arabic Speech Recognition System Based on MFCC and HMMs
  https://www.scirp.org/pdf/jcc_2020030409475461.pdf
- Speech Recognition Challenge in the Wild: Arabic MGB-3
  https://arxiv.org/pdf/1709.07276v1.pdf
- Machine Learning for Arabic Text To Speech Synthesis: a Tacotron Approach
  https://ceur-ws.org/Vol-2870/paper6.pdf
- Perceiver: One Neural-Network Model for Multiple Input Data Types
  https://www.infoq.com/news/2021/04/perceiver-neural-network-model/
- HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis
  https://arxiv.org/pdf/2010.05646.pdf
- XTTS v2: new version of the open-source text-to-speech model.
  https://medium.com/machine-learns/xtts-v2-new-version-of-the-open-source-text-to-speech-model-af73914db81f

- XTTS: a Massively Multilingual Zero-Shot Text-to-Speech Model
  https://arxiv.org/pdf/2006.04558.pdf
- Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech
  https://arxiv.org/pdf/2307.03917.pdf
- YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for everyone
  https://arxiv.org/pdf/2303.11816.pdf
- Tacotron: Towards End-to-End Speech Synthesis
  https://aclanthology.org/2023.arabicnlp-1.5.pdf
- SpeedySpeech: Efficient Neural Speech Synthesis
  https://arxiv.org/abs/2008.03802
- Effective Use of Variational Embedding Capacity in Expressive End-to-End Speech Synthesis
  https://arxiv.org/abs/1906.03402
- In defence of metric learning for speaker recognition
  https://arxiv.org/pdf/2003.11982
- UnivNet: A Neural Vocoder with Multi-Resolution Spectrogram Discriminators for High-Fidelity Waveform Generation
  https://arxiv.org/abs/2106.07889
- WaveGrad: Estimating Gradients for Waveform Generation
  https://arxiv.org/abs/2009.00713
- High Fidelity Speech Synthesis with Adversarial Networks
  https://arxiv.org/abs/1909.11646
- High Fidelity Speech Synthesis with Adversarial Networks
  https://arxiv.org/abs/1909.11646

PROJECT STAGES

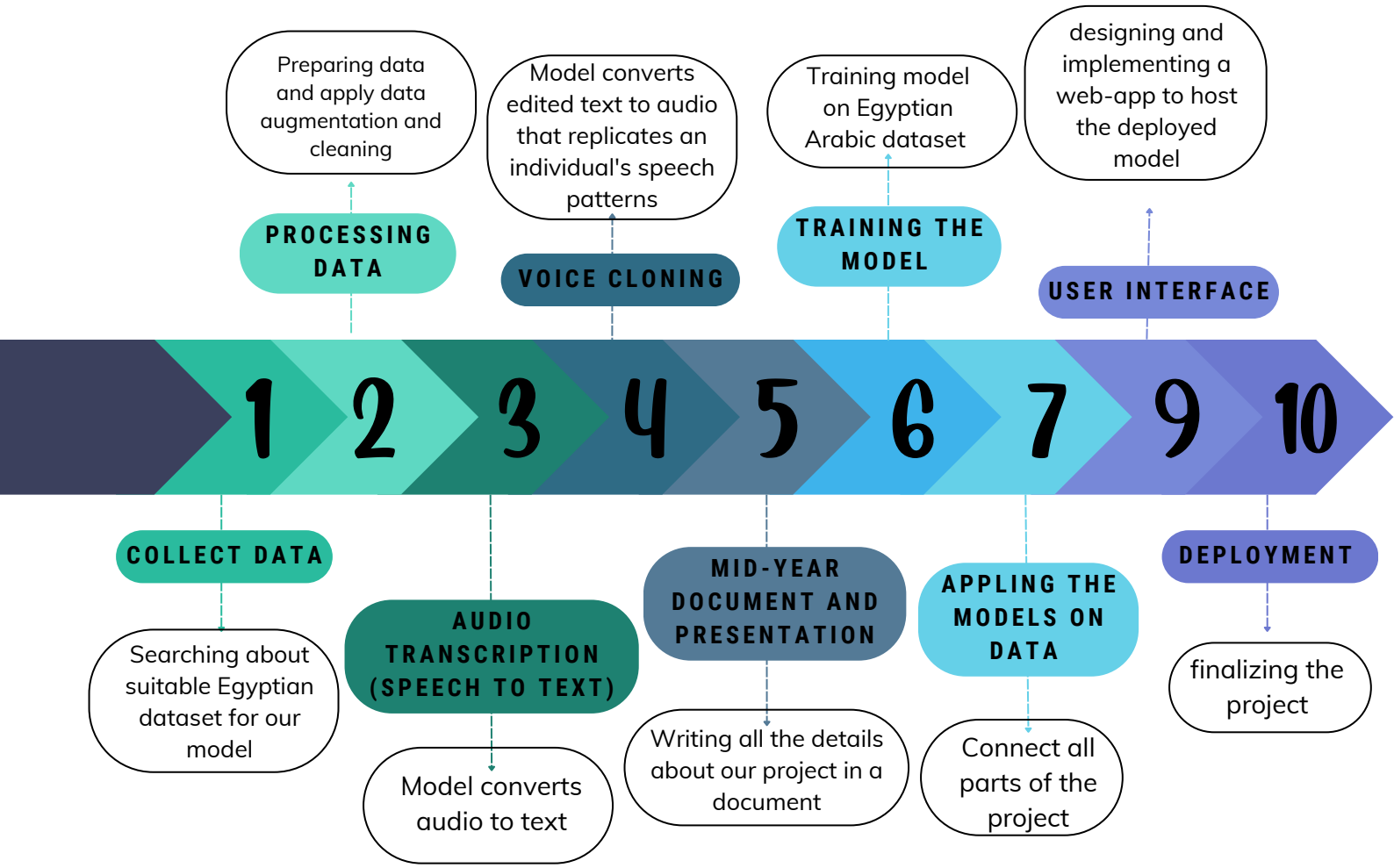

Figure 15

PLANE ACHIEVEMENT TILL MID-PROJECT

1	COLLECT DATA	Searching about suitable Egyptian dataset for our model	Done
2	PROCESSING DATA	Preparing data and apply data augmentation and cleaning	Work on
3	AUDIO TRANSCRIPTION (SPEECH TO TEXT)	Model converts audio to text	Done
4	VOICE CLONING	Model converts edited text to audio that replicates an individual's speech patterns	Done
5	MID-YEAR DOCUMENT AND PRESENTATION	Writing all the details about our project in a document	Done
6	TRAINING THE MODEL	Training model on Egyptian Arabic dataset	Work on
7	APPLING THE MODELS ON DATA	Connect all parts of the project	Planned
8	USER INTERFACE	designing and implementing a web-app to host the deployed model	Planned
9	DEPLOYMENT	finalizing the project	Planned

Figure 16

FINAL PLANE ACHIEVEMENT

1	COLLECT DATA	Searching about suitable Egyptian dataset for our model	Done
2	PROCESSING DATA	Preparing data and apply data augmentation and cleaning	Done
3	AUDIO TRANSCRIPTION (SPEECH TO TEXT)	Model converts audio to text	Done
4	VOICE CLONING	Model converts edited text to audio that replicates an individual's speech patterns	Done
5	MID-YEAR DOCUMENT AND PRESENTATION	Writing all the details about our project in a document	Done
6	TRAINING THE MODEL	Training model on Egyptian Arabic dataset	Done
7	APPLING THE MODELS ON DATA	Connect all parts of the project	Done
8	USER INTERFACE	designing and implementing a web-app to host the deployed model	Done
9	DEPLOYMENT	finalizing the project	Done

Figure 17